

Smart Room Monitoring System

Documentation

Team A4

Date: 01.07.2026

1. Team members

Andrey Sysoev - andrey.sysoev@stud.hshl.de

Sadim Rahman Badhan - sadim-rahman.badhan@stud.hshl.de

Md Mamun Hossain - md-mamun.hossain@stud.hshl.de

Umut Karakaya - umut-faruk.karakaya@stud.hshl.de

2. Introduction

The goal of this project is to monitor the physical environment of a room by using sensors, transfer them via wireless communication and a display with dashboard. The system measures temperature, humidity, light level and sound level. The measured values are transmitted wirelessly to a Raspberry Pi, stored in a SQLite and displayed on a web-based display.

The project is connected to the Internet of Things (**IoT**), because it involves physical layer with a network, which uses collected data to be transferred and visualized. In this system, the Arduino Uno WiFi Rev2 and ESP32 act as sensor nodes. They read the connected sensors and publish the measured values as MQTT messages.

The project also follows the idea of a Wireless Sensor Network (**WSN**): system have transceivers that collect environmental data and send it to another device. In this prototype, the sensor nodes send data over Wi-Fi using the MQTT protocol. The Raspberry Pi works as the receiver and stores the incoming data.

The target application is indoor room monitoring for a small smart home environment. The system allows the user to check whether the room is humid, bright or noisy without directly accessing the sensors, MQTT broker or database.

3. Concept description

The prototype follows:

Sensors → Arduino/ESP32 → MQTT Broker → Raspberry Pi → SQLite Database → Dashboard

The sensors measure the room conditions. The Arduino/ESP32 reads the sensor values and publishes them as MQTT messages. The MQTT broker forwards the messages to the Raspberry Pi. The Raspberry Pi receives the data, processes it and stores it in a SQLite database. The dashboard shows the latest sensor values to the user.

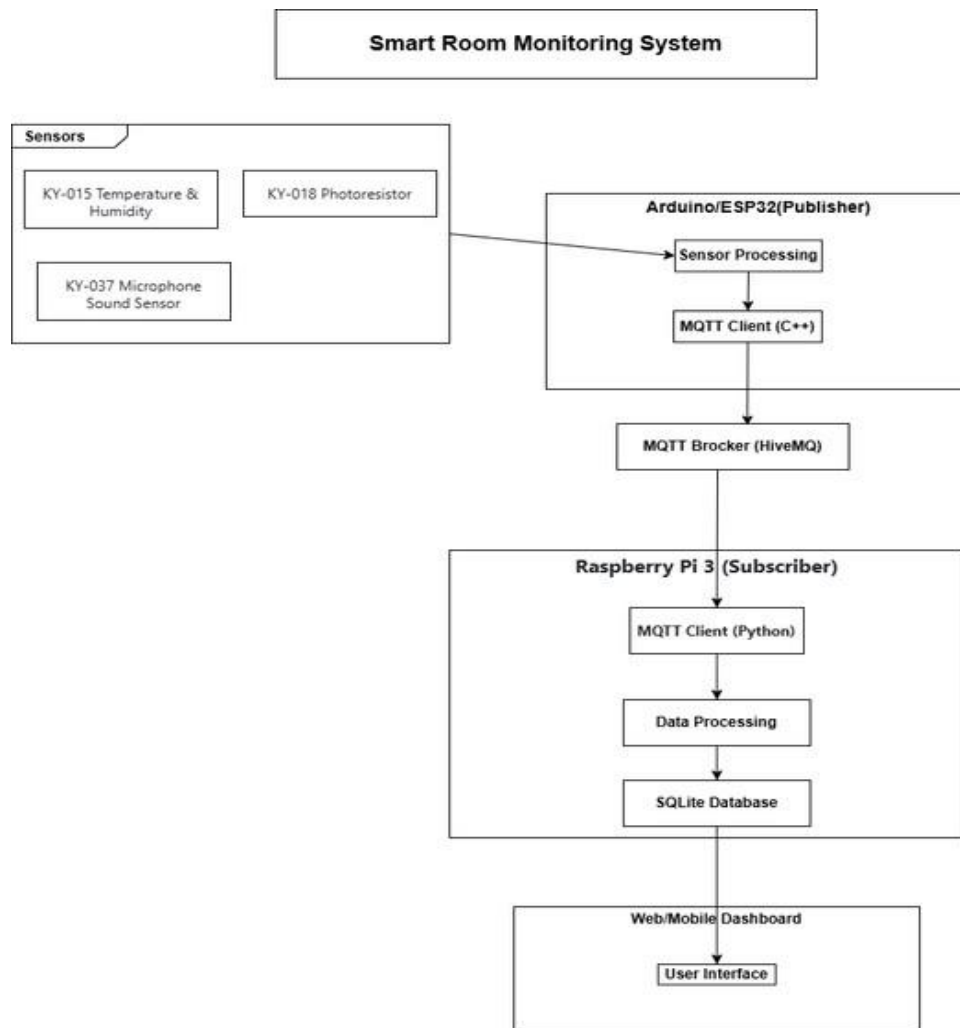


Figure 1: Block diagram of the Smart Room Monitoring System

The system uses three sensor components:

- KY-015 sensor measures temperature and humidity
- KY-018 photoresistor measures the light level
- KY-037 microphone sensor measures the sound level

The light and sound values are displayed as relative values. Therefore, they are shown as percentage-like values from 0 to 100. Temperature is displayed in Celsius and humidity is displayed in percent.

HiveMQ is used as the MQTT broker. The Raspberry Pi 3 works as the MQTT subscriber. It receives the sensor messages from the broker and stores them in the SQLite database. The dashboard is the user interface of the system. The user can view the room conditions without directly accessing the hardware, MQTT broker or database.

The block diagram shows the communication order. First, the sensors provide temperature, humidity, light and sound readings. Then the Arduino/ESP32 publishes MQTT messages. The broker forwards these messages to the Raspberry Pi. The Raspberry Pi inserts the values into the database. After that, the dashboard can request and display the latest sensor data.

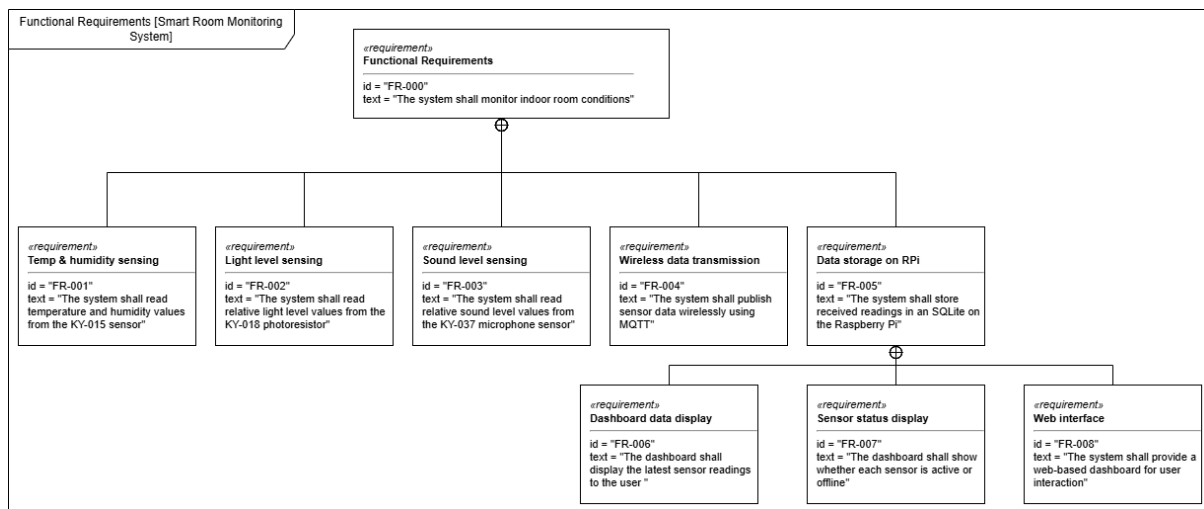


Figure 2: Functional Req. diagram of the Smart Room Monitoring System

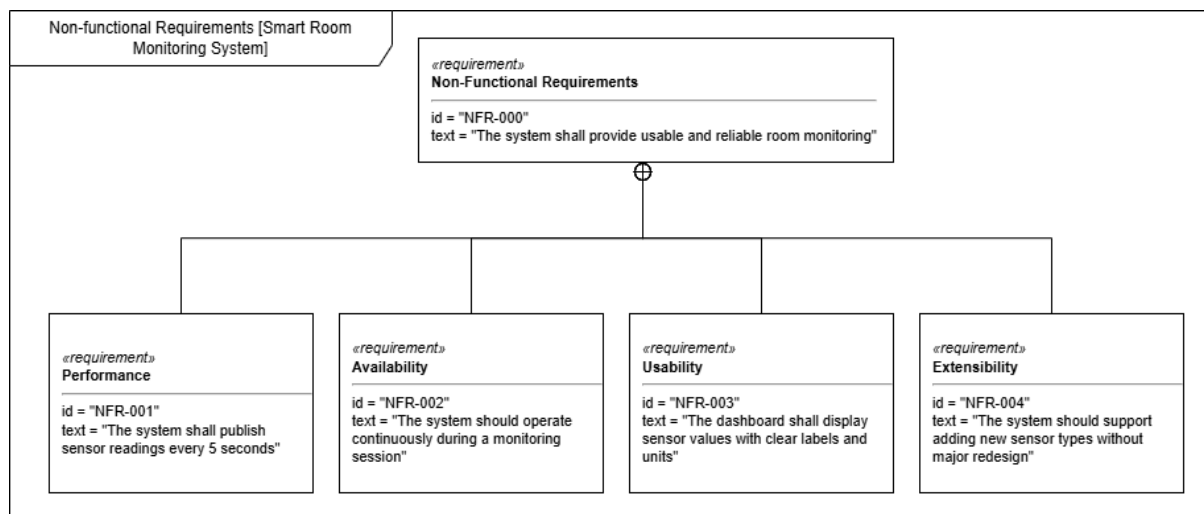


Figure 3: Non-functional Req. diagram of the Smart Room Monitoring System

4. Project/Team management

The project was divided into smaller parts to make the work easier to organize. The main parts were sensor setup, Arduino/ESP32 programming, MQTT communication, Raspberry Pi subscriber implementation, database handling, dashboard development, diagrams and documentation. The team first defined the project idea and the system requirements. After that, the system architecture was planned with block diagrams, requirement diagrams, a sequence diagram and a use case diagram.

The implementation was done step by step. First, the sensors were connected and tested. Then the Arduino/ESP32 was prepared to publish sensor values through MQTT. After that, the Raspberry Pi subscriber was implemented in Python. The subscriber receives MQTT messages, reads the JSON payload and stores the data in the SQLite database. Finally, the dashboard was used to display the room monitoring values.

GitHub was used for code and file management. The team used an iterative method, where each module was tested separately before combining it with the full system.

Member	Role in project
Andrey Sysoev	Arduino/ESP32 programming, MQTT implementation, sensor-side testing, dashboard testing, documentation correction and revision, diagram correction.
Sadim Rahman Badhan	Arduino programming, MQTT implementation, Raspberry Pi client implementation, sensor testing, database handling and dashboard development.
Md Mamun Hossain	Diagrams, documentation, MQTT communication and sensor testing support for the KY-015 and KY-018 sensors
Umut Karakaya	Diagrams, documentation, MQTT communication and Arduino communication support.

5. Technologies

5.1 Hardware and sensors

This project include Arduino Uno Wi-Fi Rev2 boards, an ESP32 development board, Sensor Kit modules and a Raspberry Pi 3 Model B. The software side includes MQTT communication, a Python subscriber, an SQLite database and a Flask-based web dashboard.

The system uses three sensor modules:

- The KY-015 sensor measures temperature and humidity
- The KY-018 photoresistor is used for light level measurement
- The KY-037 microphone sensor is used for sound level measurement.

5.2 Communication and data storage

The Arduino/ESP32 sensor nodes read the connected sensors and publish the measured values as JSON messages. The messages are sent over Wi-Fi using the MQTT protocol. MQTT is used because it is lightweight and suitable for IoT and WSN systems. The Arduino/ESP32 acts as the MQTT publisher, while the Raspberry Pi acts as the MQTT subscriber. HiveMQ is used as the MQTT broker and forwards the messages between publisher and subscriber.

5.3 Dashboard and tools

The dashboard is implemented as a Flask-based web interface. It reads the latest values from the SQLite database and displays temperature, humidity, relative light level and relative sound level in a readable way. The dashboard also shows whether each sensor is currently active or offline.

The main tools used in the project are Arduino IDE, Arduino C/C++, Python, Paho MQTT, MQTT, HiveMQ, SQLite, Flask, Bash scripts, GitHub and draw.io.

6. Implementation

The Smart Room Monitoring System was implemented using several components. The hardware setup consists of two Arduino Uno WiFi Rev2 boards, one ESP32 development board, one Raspberry Pi 3, and three Sensor Kit modules: the KY-015 temperature and humidity sensor, the KY-018 photoresistor, and the KY-037 microphone sound sensor.

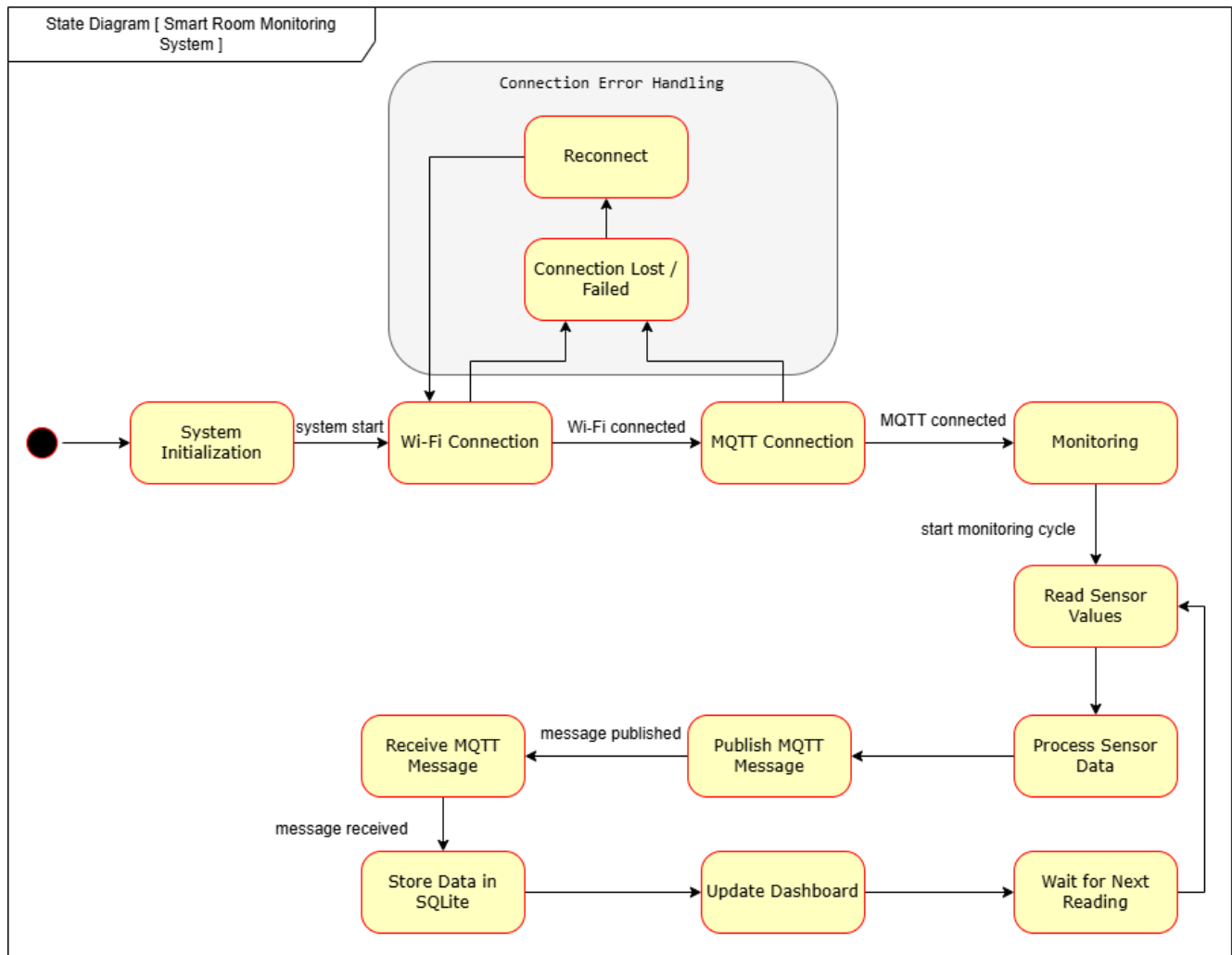


Figure 4: State diagram of the Smart Room Monitoring System

After startup, the system initializes the hardware, connects to Wi-Fi and establishes the MQTT connection. After that, it enters the monitoring state.

During monitoring, the sensor values are read and published as MQTT messages. The Raspberry Pi receives the messages, stores the data in the SQLite database and the dashboard displays the latest readings. This cycle is repeated approximately every five seconds. If the Wi-Fi or MQTT connection is lost or fails, the system enters the reconnect state. After the connection is restored, the system continues the normal cycle.

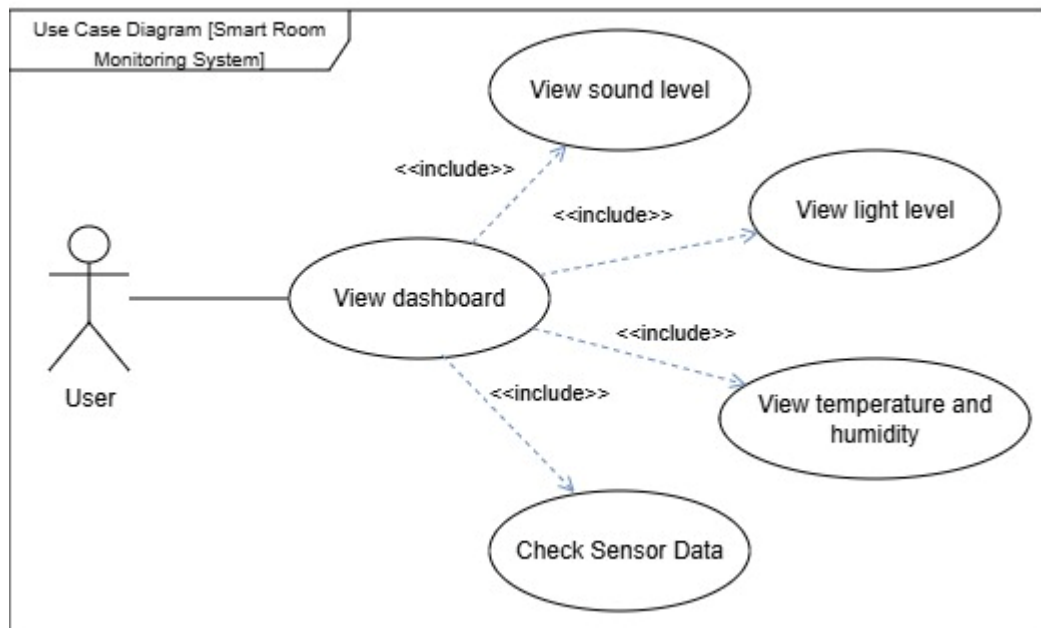


Figure 5: Use case diagram of the Smart Room Monitoring System

Figure 5 shows the interaction of the Smart Room Monitoring System: user accesses the dashboard to monitor the current room conditions. The dashboard shows temperature and humidity readings, relative light level, relative sound level and the current status of each sensor.

7. Results

The complete data path was tested from the sensor nodes to the web dashboard. During testing, the Arduino/ESP32 nodes published the sensor readings through MQTT, while the Raspberry Pi subscriber received the messages and stored them in the SQLite database. The dashboard then displayed the most recent values together with the sensor status.

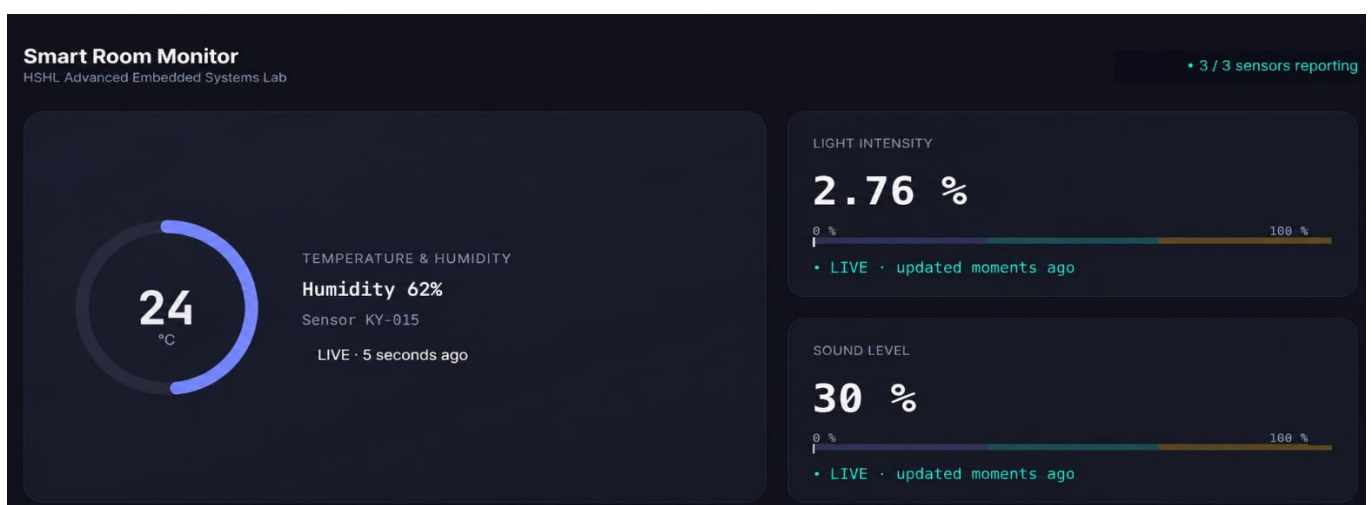


Figure 6: User Interface

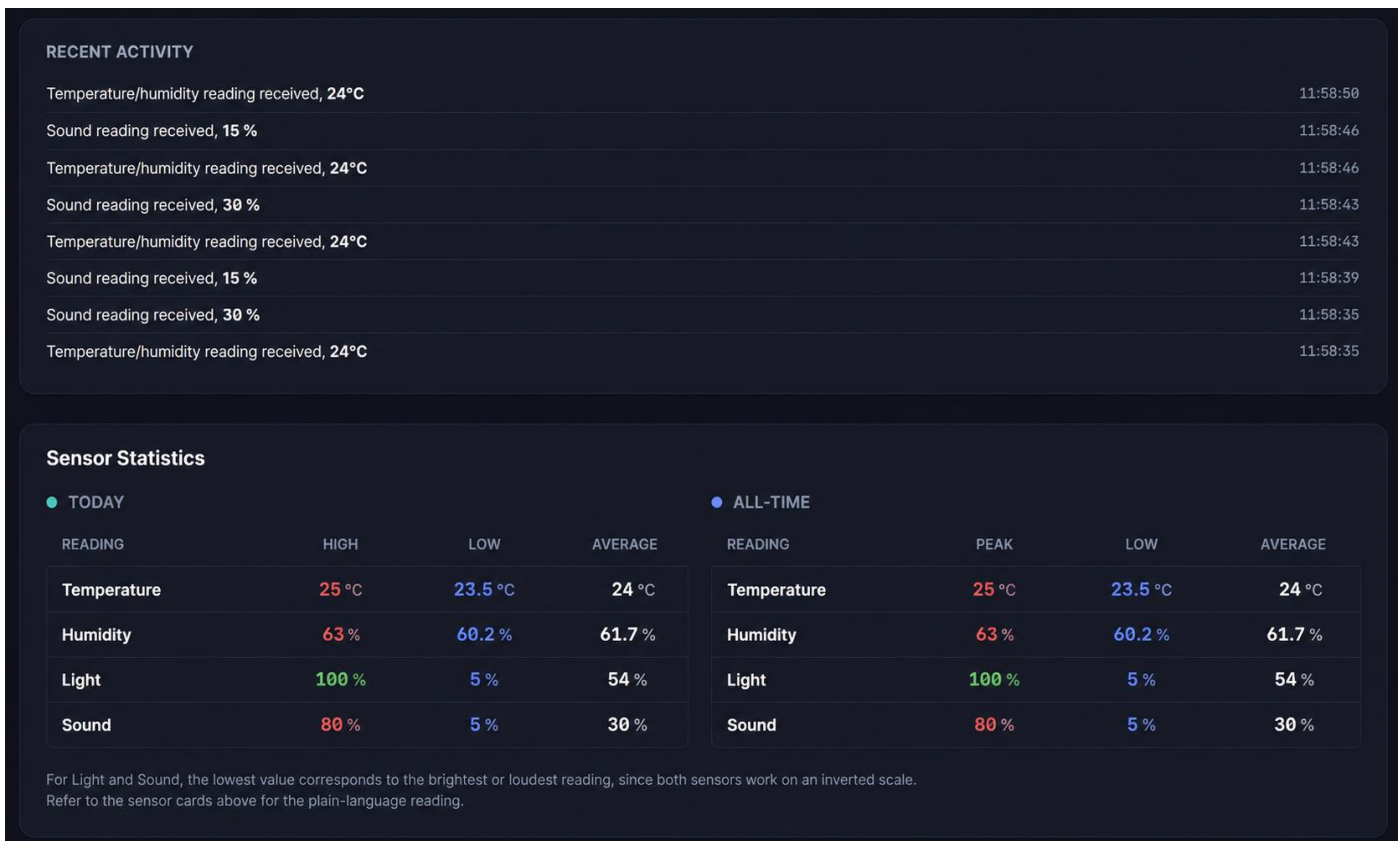


Figure 7: Activity log and sensor statistics

- **End-to-end data flow.** Sensor values were read by the Arduino/ESP32 nodes, transmitted through MQTT, received by the Raspberry Pi and stored in the SQLite database. The same values were then shown on the Flask dashboard.
- **Dashboard operation.** The dashboard updated the visible room values and showed the current state of the sensors. This allowed the user to monitor the room without opening the MQTT broker, the database or the serial monitor.
- **Sensor behavior.** The KY-015 sensor provided stable temperature and humidity readings during the test. The KY-018 photoresistor reacted to changes in light conditions and the KY-037 microphone sensor provided relative sound level values.
- **Data history.** The activity log and statistics showed that the system was not only displaying one current value, but also keeping recent measurement data in the database.

8. Sources/References

- [1] Arduino Documentation, "Arduino Uno WiFi Rev2",
<https://docs.arduino.cc/hardware/uno-wifi-rev2>
- [2] Arduino Documentation, "ArduinoMqttClient Library",
<https://docs.arduino.cc/libraries/arduinomqttclient>
- [3] Raspberry Pi Ltd., "Raspberry Pi 3 Model B",
<https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>
- [4] Eclipse Foundation, "Eclipse Paho MQTT Python Client Documentation",
<https://eclipse.dev/paho/files/paho.mqtt.python/html/>
- [5] HiveMQ, "MQTT Essentials",
<https://www.hivemq.com/mqtt/>
- [6] Flask Documentation, "Welcome to Flask",
<https://flask.palletsprojects.com/>
- [7] SQLite Documentation,
<https://sqlite.org/docs.html>
- [8] SensorKit Joy-IT, "KY-015 Combi-Sensor Temperature & Humidity",
<https://sensorkit.joy-it.net/en/sensors/ky-015>
- [9] SensorKit Joy-IT, "KY-018 Photoresistor",
<https://sensorkit.joy-it.net/en/sensors/ky-018>
- [10] SensorKit Joy-IT, "KY-037 Microphone Sound Sensor",
<https://sensorkit.joy-it.net/en/sensors/ky-037>
- [11] GitHub Repository, "SS2026_AES_Lab_A4",
https://github.com/andrey-sysoev/SS2026_AES_Lab_A4